

Universidad de Costa Rica

IE0435 – Inteligencia Artificial Aplicada a la Ingeniería Eléctrica

Proyecto 1

Clasificación de Contaminaciones en una Línea de Producción Simulada

Estudiante:

Felipe Ulate Rodríguez

Carné: C37925

Índice

1. Introducción	2
2. Objetivo	2
3. Descripción del conjunto de datos	2
4. Preprocesamiento de imágenes	3
5. Conformación del conjunto de datos final	3
6. Modelos evaluados	4
7. Resultados	4
8. Exportación del modelo	6
9. Repositorio y reproducibilidad	6
10.Limitaciones	6
11.Conclusiones	7
12.Referencias	7

1. Introducción

En los procesos de manufactura, la inspección visual permite detectar defectos, contaminaciones o anomalías que pueden afectar la calidad del producto. Tradicionalmente, este tipo de inspección puede depender de observación humana, lo cual introduce variabilidad, fatiga y posibles errores. Por esta razón, el uso de técnicas de visión artificial y aprendizaje automático representa una alternativa útil para apoyar la detección automática de elementos no deseados.

El presente proyecto consiste en la clasificación de contaminaciones en una línea de producción simulada. En esta simulación se utilizaron imágenes tomadas sobre una hoja blanca, donde la presencia de granos de arroz representa una contaminación positiva. Por otra parte, imágenes sin arroz, vacías o con otros objetos se consideran muestras negativas.

El trabajo desarrollado incluye el procesamiento de imágenes, la conversión de estas en matrices binarias, la conformación de un conjunto de datos grupal, la evaluación de diferentes modelos clásicos de clasificación y la exportación del mejor modelo obtenido.

2. Objetivo

Desarrollar un sistema de clasificación basado en aprendizaje automático clásico para identificar la presencia de granos de arroz como contaminaciones positivas en imágenes de una línea de producción simulada.

3. Descripción del conjunto de datos

El conjunto de datos fue construido a partir de imágenes tomadas sobre una hoja blanca. Las imágenes fueron clasificadas en dos grupos:

- **Clase 1:** imágenes positivas, es decir, imágenes con presencia de granos de arroz.
- **Clase 0:** imágenes negativas, es decir, imágenes sin arroz, vacías o con presencia de otros objetos.

Inicialmente, las imágenes propias fueron procesadas para generar el archivo `matriz_final.csv`. Posteriormente, este archivo fue integrado con archivos CSV generados por otros compañeros del grupo. La unificación de estos archivos permitió construir un conjunto de datos consolidado para entrenar y evaluar modelos de clasificación.

Cada imagen procesada fue transformada en un vector fila compuesto por 16384 valores de píxeles y una columna final correspondiente a la etiqueta. Esto se debe a que cada imagen fue redimensionada a:

$$128 \times 128 = 16384 \text{ píxeles}$$

Por tanto, cada fila del conjunto de datos final contiene:

$$16384 \text{ píxeles} + 1 \text{ etiqueta} = 16385 \text{ columnas}$$

4. Preprocesamiento de imágenes

El preprocesamiento de imágenes se realizó mediante el script `proyecto_arroz.py`. Este script automatiza la conversión de imágenes en datos numéricos aptos para ser utilizados por modelos de aprendizaje automático.

El procedimiento aplicado fue el siguiente:

1. Lectura de imágenes desde las carpetas `Positivo` y `Negativo`.
2. Conversión de cada imagen a escala de grises.
3. Redimensionamiento de cada imagen a **128x128 píxeles**.
4. Aplicación de binarización mediante el método de Otsu.
5. Conversión de la imagen binaria a una matriz de unos y ceros.
6. Vectorización de la matriz de píxeles.
7. Adición de una última columna con la etiqueta de clasificación.

La codificación utilizada para los píxeles fue:

- **1:** píxel blanco o fondo.
- **0:** píxel con presencia de objeto.

La etiqueta final de cada imagen se codificó de la siguiente manera:

- **1:** imagen con presencia de arroz.
- **0:** imagen sin presencia de arroz.

5. Conformación del conjunto de datos final

La conformación del conjunto de datos grupal se realizó mediante el script `conjunto_datos.py`. Este script permitió unir varios archivos CSV generados individualmente por los integrantes del grupo.

El script realiza las siguientes acciones:

- Carga todos los archivos CSV ubicados en la carpeta de datos.
- Detecta automáticamente el separador utilizado en cada archivo.

- Elimina posibles encabezados accidentales.
- Convierte todos los valores a formato numérico.
- Verifica que cada archivo tenga 16385 columnas.
- Comprueba que los valores presentes sean únicamente 0 y 1.
- Une los archivos válidos en un único conjunto de datos final.
- Guarda el dataset consolidado sin encabezados y sin índice.

Este proceso permitió estandarizar los archivos individuales y asegurar que todos tuvieran el mismo formato antes de ser utilizados para la evaluación de modelos.

6. Modelos evaluados

Para la etapa de clasificación se evaluaron diferentes modelos clásicos de aprendizaje automático mediante el script `eval_modelos.py`. Los modelos evaluados fueron:

- SVM con kernel lineal.
- SVM con kernel RBF.
- KNN con diferentes valores de k .
- Naive Bayes.
- Árbol de Decisión.
- Árbol de Decisión con profundidad máxima de 5.
- Random Forest.

La evaluación se realizó utilizando una partición del conjunto de datos en una proporción de **80 % para entrenamiento** y **20 % para prueba**. Además, se utilizó estratificación por clase para mantener el balance entre muestras positivas y negativas. Para garantizar reproducibilidad, se empleó `random_state=42`.

7. Resultados

Los resultados obtenidos para cada modelo se presentan en la Tabla 1.

Cuadro 1: Resultados de accuracy para los modelos evaluados.

Modelo	Accuracy
SVM Lineal	0.5667
SVM RBF	0.5333
KNN k=3	0.6000
KNN k=5	0.5667
KNN k=7	0.5000
Naive Bayes	0.4000
Árbol de Decisión	0.7000
Árbol de Decisión max_depth=5	0.6333
Random Forest	0.6000

El modelo con mejor desempeño fue el **Árbol de Decisión**, con un accuracy de **0.7000**. Esto significa que el modelo clasificó correctamente el 70 % de las muestras del conjunto de prueba.

La matriz de confusión obtenida para el mejor modelo fue:

$$\begin{bmatrix} 13 & 2 \\ 7 & 8 \end{bmatrix}$$

Esta matriz indica que el modelo clasificó correctamente 13 muestras de la clase 0 y 8 muestras de la clase 1. También se observaron 2 falsos positivos y 7 falsos negativos.

Cuadro 2: Reporte de clasificación del Árbol de Decisión.

Clase	Precision	Recall	F1-score	Support
0	0.65	0.87	0.74	15
1	0.80	0.53	0.64	15
Accuracy			0.70	30
Macro avg	0.73	0.70	0.69	30
Weighted avg	0.72	0.70	0.69	30

A partir del reporte de clasificación, se observa que el modelo tuvo mejor desempeño identificando la clase negativa que la clase positiva. Esto se refleja en el recall de la clase 0, que fue de 0.87, mientras que el recall de la clase 1 fue de 0.53. Por tanto, el modelo presenta mayor dificultad para detectar algunas imágenes con presencia de arroz.

8. Exportación del modelo

El mejor modelo obtenido fue exportado en formato `.joblib` mediante la biblioteca `joblib`. Este formato permite guardar el modelo entrenado para utilizarlo posteriormente sin necesidad de volver a ejecutar el proceso de entrenamiento.

El modelo fue guardado en la carpeta `models/` del repositorio. Para ajustarse al formato solicitado, el archivo puede nombrarse de la siguiente manera:

```
C37925_Felipe_Ulate_Rodriguez.joblib
```

9. Repositorio y reproducibilidad

El proyecto fue documentado en un repositorio de GitHub, donde se incluyen los scripts utilizados, los datos procesados, el modelo exportado y los archivos de documentación solicitados.

El repositorio contiene la siguiente estructura general:

```
Proyecto1_IE0435/  
  data/  
  models/  
  reports/  
  src/  
  DATASET.md  
  MODEL_CARD.md  
  README.md  
  requirements.txt  
  LICENSE
```

El enlace al repositorio es:

https://github.com/feliulater/Proyecto1_IE0435

Para reproducir el flujo completo del proyecto, se deben ejecutar los scripts en el siguiente orden:

```
python src/proyecto_arroz.py  
python src/conjunto_datos.py  
python src/eval_modelos.py
```

Las dependencias necesarias para ejecutar el proyecto se encuentran en el archivo `requirements.txt`.

10. Limitaciones

El proyecto presenta algunas limitaciones que pueden afectar el desempeño de los modelos:

- El tamaño del conjunto de datos es reducido.
- Existen variaciones de iluminación entre imágenes.
- Las imágenes pueden haber sido capturadas con diferentes cámaras.
- Pueden existir sombras o ruido sobre el fondo blanco.
- Algunos objetos pequeños pueden presentar formas similares a los granos de arroz.
- El modelo utiliza directamente píxeles binarios, sin extracción avanzada de características.
- El accuracy obtenido es moderado, por lo que todavía existe margen de mejora.

Estas limitaciones explican por qué algunos modelos no lograron un desempeño alto y por qué el Árbol de Decisión, aunque fue el mejor modelo evaluado, todavía presenta errores de clasificación.

11. Conclusiones

Se logró construir un flujo completo para la clasificación de contaminaciones en una línea de producción simulada. Este flujo incluyó la recolección y procesamiento de imágenes, la conversión de estas a matrices binarias, la conformación de un conjunto de datos grupal, la evaluación de modelos clásicos de aprendizaje automático y la exportación del mejor modelo obtenido.

El modelo con mejor desempeño fue el Árbol de Decisión, alcanzando un accuracy de 0.7000 en la partición de prueba utilizada. Este resultado es aceptable para una primera aproximación académica, considerando el tamaño reducido del conjunto de datos y las posibles variaciones en iluminación, cámara y posición de los objetos.

Sin embargo, el análisis del reporte de clasificación muestra que el modelo tuvo mayor dificultad para identificar correctamente algunas muestras positivas. Por esta razón, futuras mejoras podrían incluir una mayor cantidad de imágenes, condiciones de captura más controladas, extracción de características visuales y evaluación con más particiones o validación cruzada.

12. Referencias

- Material del Proyecto 1 del curso IE0435: Inteligencia Artificial Aplicada a la Ingeniería Eléctrica.
- Documentación de OpenCV para procesamiento de imágenes.
- Documentación de Scikit-learn para modelos clásicos de clasificación.